Centro Paula Souza

Faculdade de Tecnologia de Votorantim

Curso de Ciência de Dados para Negócios

Aluno: Ana Elisa Rubinato Cavalheiro

Estatística

*Logs e Depuração *

```
import logging
from scipy.stats import linregress
import numpy as np
import os

# Definir caminho absoluto para o log
log_path = "/content/log_trendline.log"
# Apagar arquivo de log anterior (opcional para testes)
if os.path.exists(log_path):
    os.remove(log_path)
    print(f"Arquivo de log apagado: {log_path}")
# Criar logger totalmente novo e isolado
logger = logging.getLogger("trendline_logger")
logger.setLevel(logging.INFO)
# Limpar handlers anteriores para evitar conflito no Colab
if logger.hasHandlers():
    logger.handlers.clear()
# Criar handler de arquivo
file_handler = logging.FileHandler(log_path, mode='w') # 'w' sobrescreve
file_handler.setLevel(logging.INFO)
# Definir formatação
formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
file_handler.setFormatter(formatter)
# Adicionar handler ao logger
logger.addHandler(file_handler)


# Função de regressão
def fit_trendline(year_timestamp, data):
    logger.info("Executando a função fit_trendline")
    result = linregress(year_timestamp, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    logger.info(f"Análise completa. Slope da função trendline é {slope}")
    return slope, r_squared

# Dados de exemplo
anos = np.array([2015, 2016, 2017, 2018, 2019, 2020])
valores = np.array([100, 120, 130, 150, 170, 200])

# Executar função
slope, r2 = fit_trendline(anos, valores)
print("Coeficiente angular:", slope)
print("Coeficiente de determinação R²:", r2)

# Forçar gravação no disco
for handler in logger.handlers:
    handler.flush()
print(f"Arquivo de log gravado: {log_path}")

# Ler o log
print("\nConteúdo do log:")
with open(log_path, "r") as f:
    print(f.read())
```

```
INFO:trendline_logger:Executando a função fit_trendline
INFO:trendline_logger:Análise completa. Slope da função trendline é 19.143
Coeficiente angular: 19.143
Coeficiente de determinação R²: 0.979
Arquivo de log gravado: /content/log_trendline.log

Conteúdo do log:
2025-06-03 18:19:54,200 - INFO - Executando a função fit_trendline
2025-06-03 18:19:54,217 - INFO - Análise completa. Slope da função trendline é 19.143
```

```
def calcular_media(notas):
    soma = 0
```

```
    for nota in notas:
        soma += nota
        print(f"DEBUG: Soma atual = {soma} depois de adicionar {nota}") # Ponto de depuração
    media = soma / len(notas)
    print(f"DEBUG: Média final = {media}") # Ponto de depuração
    return media

notas_alunos = [7, 8, 5, 9, 6]
media_final = calcular_media(notas_alunos)
print(f"A média das notas é: {media_final}")

# Exemplo com um erro sutil: uma nota que não deveria ser adicionada
notas_com_erro = [7, 8, 5, "dez", 6] # "dez" causará um TypeError
try:
    media_com_erro = calcular_media(notas_com_erro)
    print(f"A média das notas com erro é: {media_com_erro}")
except TypeError as e:
    print(f"Erro capturado: {e}. Isso aconteceu porque tentamos somar um tipo inválido.")
```

```
DEBUG: Soma atual = 7 depois de adicionar 7
DEBUG: Soma atual = 15 depois de adicionar 8
DEBUG: Soma atual = 20 depois de adicionar 5
DEBUG: Soma atual = 29 depois de adicionar 9
DEBUG: Soma atual = 35 depois de adicionar 6
DEBUG: Média final = 7.0
A média das notas é: 7.0
DEBUG: Soma atual = 7 depois de adicionar 7
DEBUG: Soma atual = 15 depois de adicionar 8
DEBUG: Soma atual = 20 depois de adicionar 5
Erro capturado: unsupported operand type(s) for +=: 'int' and 'str'. Isso aconteceu porque tentamos somar um tipo inválido.
```

```
# !pip install ipdb # Instale o pacote ipdb se ainda não estiver instalado
!pip install ipdb
```

```
Collecting ipdb
  Downloading ipdb-0.13.13-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: ipython>=7.31.1 in /usr/local/lib/python3.11/dist-packages (from ipdb) (7.34.0)
Requirement already satisfied: decorator in /usr/local/lib/python3.11/dist-packages (from ipdb) (4.4.2)
Requirement already satisfied: setuptools>=18.5 in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (75.2.0)
Collecting jedi>=0.16 (from ipython>=7.31.1->ipdb)
  Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: pickleshare in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (0.7.5)
Requirement already satisfied: traitlets>=4.2 in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (5.7.1)
Requirement already satisfied: prompt-toolkit!=3.0.0,!=3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from ipythor
Requirement already satisfied: pygments in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (2.19.1)
Requirement already satisfied: backcall in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (0.2.0)
Requirement already satisfied: matplotlib-inline in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (0.1.7)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-packages (from ipython>=7.31.1->ipdb) (4.9.0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/python3.11/dist-packages (from jedi>=0.16->ipython>=7.31.1->ipd
Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.11/dist-packages (from pexpect>4.3->ipython>=7.31.1->ipdb)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-packages (from prompt-toolkit!=3.0.0,!=3.0.1,<3.1.0,>=2.0.0
Downloading ipdb-0.13.13-py3-none-any.whl (12 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 43.7 MB/s eta 0:00:00
Installing collected packages: jedi, ipdb
Successfully installed ipdb-0.13.13 jedi-0.19.2
```

```
def calcular_desconto(valor_original, percentual_desconto):
    # ipdb.set_trace() # Descomente para ativar o depurador aqui
    valor_com_desconto = valor_original * (1 - percentual_desconto / 100)
    return valor_com_desconto

preco_produto = 100
desconto_aplicado = 10

def dividir(a, b):
    # ipdb.set_trace()
    return a / b

try:
    resultado = dividir(10, 0) # Isso causará um ZeroDivisionError
    print(resultado)
except ZeroDivisionError:
    print("Ocorreu um erro de divisão por zero. Entrando no depurador para investigar.")
```

```
Ocorreu um erro de divisão por zero. Entrando no depurador para investigar.
```